🔹 1. The Problem Generators Solve

❌ **Normal function**

```
def get_numbers(n):
    result = []
    for i in range(n):
        result.append(i)
    return result
```

👉 Problem:

- Stores **all values in memory**
- Slow for large data

🔹 2. Generator = Lazy Producer

A **generator** produces values **one at a time, only when needed**.

🔹 3. What yield Really Does

👉 yield turns a function into a **paused function**

Instead of:

```
return value   # ends function
```

It does:

```
yield value    # pauses function
```

🔹 4. Execution Flow (VERY IMPORTANT)

```
def demo():
    print("Start")
    yield 1
    print("Middle")
```

```
    yield 2
    print("End")
```

Run it:

```
g = demo()

next(g)
next(g)
next(g)
```

Output:

```
Start
Middle
End
```

🔥 What actually happens:

Step	What happens
g = demo()	nothing runs
next(g)	runs → stops at first yield
next(g)	resumes → stops again
next(g)	resumes → ends

👉 **It remembers where it stopped**

🔹 5. Internal Mental Model

Think of generator like:

Function + Memory + Pause Button

It stores:

- current line

- variable values
- loop state

🔹 6. Real Example (Your Batch Generator)

```
def batch_generator(data, batch_size):
    for i in range(0, len(data), batch_size):
        yield data[i:i+batch_size]
```

Flow:

next() → [0–2]
next() → [3–5]
next() → [6–7]

👉 It **does NOT create all batches at once**

7. Why Generators Are Needed

✅ 1. Memory Efficiency

```
def big():
    for i in range(1_000_000_000):
        yield i
```

👉 Uses almost **no extra memory**

✅ 2. Speed (Lazy Execution)

- Values created **only when needed**
- No unnecessary computation

✅ 3. Infinite Sequences

```
def infinite():
    i = 1
    while True:
```

```
    yield i
    i += 1
```

👉 Impossible with normal lists ❌

✅ **4. Cleaner than Iterators**

Instead of writing:

```
class MyIter:
  def __iter__(self): ...
  def __next__(self): ...
```

You just write:

```
def gen():
  yield ...
```

🔹 8. How to Use Generators

✅ **Method 1: next()**

```
g = batch_generator([1,2,3,4], 2)

print(next(g))
print(next(g))
```

✅ **Method 2: for loop (most common)**

```
for batch in batch_generator(data, 3):
  print(batch)
```

👉 for automatically uses next()

✅ **Method 3: Convert to list**

```
list(batch_generator(data, 3))
```

🔹 **9. Generator Expression (Shortcut)**

```
squares = (x*x for x in range(5))
```

👉 Same as generator function but shorter

🔹 10. yield vs return (Core Difference)

Feature	return	yield
Ends function	✅	❌
Returns multiple values	❌	✅
Keeps state	❌	✅
Memory efficient	❌	✅

🔹 11. Common Mistake

```
g = batch_generator(data, 3)

print(list(g))
print(list(g))  # ❌ empty
```

👉 Generator is **exhausted after one use**

🔹 12. Real-Life Analogy

- **List** → full movie downloaded 🎬
- **Generator** → streaming Netflix 📡

You don't load everything—just what you need.

🔹 🔥 Final Intuition

👉 Generator = *"give me next value when I ask"*
👉 yield = *"pause here and continue later"*